

Study Assistant v2.0

Week 10 Mini-Project · From Retrieval-Ready to Production-Grade RAG
PG Diploma in AI-ML & Agentic AI Engineering · Cohort 1 · IIT Gandhinagar

A NOTE BEFORE YOU BEGIN

Last week you built the foundation. This week you build what sits on top of it: dense embeddings, vector retrieval, grounded generation, real evaluation, and the first production-shaped failure fixes (rerank, query enhancement, structure-aware chunking).

You are picking up the project two days after Major Exam 1. Your starting state is not the same as the student next to you. Some of you shipped a working Wk9 v1; some of you have a partial implementation; some of you have a notebook that runs but produces noise. That is fine. Wk10 is designed around exactly that reality.

The single most important behaviour to practise this week is the one Rajat repeated three times in Wk9: **print the retrieved chunks before blaming the model**. Anish will say it again, framed slightly differently. Both are right. The bug is almost never where your instinct points first.

Read this entire document before you start coding. The expert hint boxes are not decoration. Each one is written to save you the specific four hours it took someone else.

EXPERT HINT · What changed between Wk9 and Wk10

Wk9 was a single tier ladder: Base → Stretch → Advanced → Open. You climbed as far as you got. Wk10 works differently. There are two starting tracks (**Core** and **Stretch**) and you self-select on Tuesday morning using a 5-question check. Track is not a final grade band. It is a starting point. Anyone who ships a working v2.0 by Sunday merges into the unified Wk11 production track on Monday. The point of the dual-track is to keep the catch-up student building *this* week, not next week.

WHAT IS IN THIS DOCUMENT

- Section 1. The scenario, continued (what's new in PariShiksha's pilot)
- Section 2. What you carry forward from Wk9
- Section 3. What this project exercises (concepts ↔ decisions)
- Section 4. Environment, data, and tooling (including OpenDataLoader-PDF)
- Section 5. The track choice (Core or Stretch)
- Section 6. Core track — five stages
- Section 7. Stretch track — five stages
- Section 8. Hot Extensions (optional, after your base ships)
- Section 9. Industry Pointers (awareness — what real teams do)
- Section 10. Expert hints
- Section 11. What "done" looks like + rubric + submission
- Section 12. Reflection questionnaire (LLM-resistant)
- Section 13. Interview-flavor scenarios
- Section 14. One closing thought

1. The scenario, continued

PariShiksha's founders saw the Wk9 prototype. The reaction was a mix of excitement and a new specific worry. The excitement: the assistant did refuse out-of-scope questions and did cite chapters. The worry: the senior science teacher tested 30 of her own questions on the prototype and 8 of them came back with answers that were *almost* right. Not wrong enough to escalate. Not right enough to ship.

Her exact phrasing in the founders' review meeting: "This is a B-grade student answering a tutorial. If a parent asked me to grade this, I'd say 'do better next time.' If a parent paid us for this, I'd refund."

The founders escalated three asks for Wk10:

- 1 **The 'almost right' problem.** When the model is given relevant context but still produces a fuzzy answer, what's the actual failure mode? Retrieval ranking? Prompt? Chunk boundary? Evaluation needs to tell us, not us guessing.
- 2 **Tables and worked examples.** NCERT chapters have worked examples with their solutions in tables. The Wk9 chunker split several of these mid-row. The teacher specifically asked: "Why does it ignore the answer column?"
- 3 **The teacher wants source highlighting.** When the assistant gives an answer, she wants to see *which paragraph in which chapter* it came from. Not "Chapter 9" — the specific 200 words. This is non-negotiable for the next demo.

These three asks map directly onto three Wk10 capabilities: (1) **evaluation that diagnoses** (not just scores), (2) **structure-aware chunking and rerank** (so worked examples and tables stay coherent and the right chunk reaches the top), (3) **citation precision** (chunk_ids with bounding boxes if you have a loader that gives them — and OpenDataLoader-PDF does).

NOTE · Why the scenario keeps tightening

Real product feedback never stays at "wow, cool prototype." It tightens into specific capabilities the next demo must show. You will see this pattern in every product job you take. Wk9's success bought Wk10's harder spec. Wk11's spec will be harder again. The cohort that ships at Wk20 is the cohort that handles each tightening without panicking.

Your role hasn't changed. You are still the sole ML engineer on the prototype team. The senior backend engineer is now busy provisioning a demo environment for the founders' next investor meeting. The teacher is grading 30 more questions per chapter for an expanded eval set, which she will share Friday afternoon. You build alongside both of them.

2. What you carry forward from Wk9

Wk10 layers on top of Wk9. We do not restart. The same repo, the same NCERT chapter(s), the same PariShiksha framing. What you built last week becomes the v0 baseline that today's work upgrades.

2.1 Your Wk9 artefacts (the foundation)

Wk9 artefact	What it gives you	What Wk10 does with it
PDF extraction code (PyMuPDF or pdfplumber)	Raw text from one or two NCERT chapters	Replaced or supplemented by OpenDataLoader-PDF for structure-aware extraction (§4)
Tokenizer comparison	Awareness that GPT-2 BPE / BERT WordPiece / T5 SentencePiece split scientific terms differently	Informs chunk-size decisions in tokens, not characters
Content-type-aware chunking (concept / example / question)	First version of metadata-rich chunks	Refined Tue with token-aware sizing + structure-aware boundaries
BM25 retriever	Working lexical retrieval with top-k	Becomes one half of hybrid retrieval (BM25 + dense) on Wed/Fri
Grounding prompt with refusal	Working ask() that refuses out-of-scope (most of the time)	Hardened Thu with citation format + tighter refusal phrasing
Eval set of 15–20 questions	Honest measurement baseline	Expanded to 12–30 Q with three axes (correctness/grounding/refusal) Thu; RAGAS-evaluated for Stretch Fri
reflection.md from Wk9	Your honest record of what broke and what you noticed	Re-read it Tuesday morning. The bugs you flagged then are the ones we fix this week.

2.2 Things Rajat told you in Wk9 that still matter

These are the principles you have already heard once. They land harder this week because you now have data to test them against. Re-reading them is not optional.

- **Retrieval-time and index-time tokenisation must match.** Especially when you add dense embeddings on Wednesday — query-side and document-side embedding models must be the same.
- **Chunk boundaries that split an example from its solution destroy retrieval.** Worked examples are this week's specific test of that principle.
- **The most common hallucination is not "model invents facts."** It is "model dutifully generates from whatever context it was given, even if that context is irrelevant." When a Wk10 query produces a wrong answer, your first move is to print top-k retrieved chunks. Always.
- **"Only answer from context" is weaker than "refuse if not in context."** The first is permissive. The second is a constraint. Test the difference explicitly Thursday.
- **Set temperature = 0 for evaluation.** Reproducibility is non-negotiable when you compare v3 to v4 scores on the same eval set.
- **The gap between good chunking and bad chunking is larger than the gap between LLMs.** This is the principle Anish opens the week with. Believe it before testing it.
- **Single-variable iteration.** When you add rerank on Friday, do not also rewrite the prompt and switch the model. Commit each change. Your git log is your experiment log.

- **Out-of-scope questions that retrieve relevant-looking chunks are the real adversarial test.** Include at least one in your eval set. Then test that your strict prompt actually refuses.

NOTE · If your Wk9 ended badly

If your Wk9 system did not work — chunks broke, retrieval failed, your eval was a guess — Tuesday morning is your reset. The Core track exists for this case. You are not behind in any way that matters by Sunday May 3. The week is designed so a clean Core finish ships a working v2.0. Two-thirds of the cohort will be on Core. That is the design, not a fallback.

3. What this project exercises

Every concept from Wk10 teaching shows up as a concrete decision you make in code. None of it is decoration. If you cannot point to where a concept lives in your repo by Friday evening, you have not exercised it.

Concept (Wk10 teaching)	Where it lands in your project
Dense embeddings (text-embedding-3-small / voyage-3 / bge-small-en)	Wed §6.3 / §7.3. You embed Tue chunks into Chroma and run top-k retrieval on your Wk9 eval questions. Stretch compares two models side-by-side.
Vector stores vs vector databases	Wed §6.3 / §7.3. Core uses Chroma (local persistent, the simplest case). Stretch indexes Chroma + one of Weaviate / Pinecone / Qdrant and benchmarks latency + recall.
Top-k retrieval mechanics, cosine similarity, score interpretation	Wed §6.3 / §7.3. The k=3 vs k=5 vs k=10 experiment makes the cost (context bloat, relevance dilution) visible. Reading similarity scores honestly matters.
Where dense retrieval loses to BM25 (acronyms, exact identifiers, numeric tokens)	Wed afternoon drill + Stretch hybrid retrieval. Two queries, same retriever choice, opposite outcomes.
Anatomy of a RAG prompt; refusal as a constraint, not a preference	Thu §6.4 / §7.4. ask() with strict prompt + chunk_id citation. Refusal triggers cleanly on out-of-scope.
Three flavors of out-of-scope (off-topic / partial / plausibly answerable)	Thu eval set composition. The third flavor is where real systems fail. Your eval must contain at least one.
Light eval (correctness / grounding / refusal-appropriateness)	Thu §6.4 / §7.4. 12-Q manual scoring on three axes. Stretch expands to 30-Q and runs RAGAS Friday.
Failure-mode catalog (synonym mismatch / lost cross-refs / multi-hop / mixed structure / ambiguous)	Fri §6.5 / §7.5. You diagnose your worst Thu eval failure into one of these five before writing any fix code.
Reranking with a cross-encoder (Cohere rerank-3 or local cross-encoder)	Fri Stretch. Same query before and after — the right chunk should promote from rank 7 to rank 1. Latency cost is visible.
Query enhancement (MultiQuery, HyDE, decomposition)	Fri Stretch. MultiQueryRetriever rewrites one query into three variants. Helps on vague phrasing; hurts on concrete queries.
Structure-aware chunking (tables, equations, code/Markdown blocks)	Fri Hot Extension. NCERT worked-example tables become the test case. Naive splitter destroys them; structure-aware keeps them whole.
Citation precision (chunk_ids, page numbers, bounding boxes)	Throughout Wk10. The teacher's ask becomes a feature: every answer returns chunk_ids; OpenDataLoader-PDF lets you go further with bounding boxes.
Production failure-mode awareness: Self-RAG / CRAG / agentic retrieval	Awareness only. Mentioned briefly Friday. Built only as a Hot Extension if your base shipped clean. Sunday's optional LangGraph teaser sketches them.

EXPERT HINT · Concept ↔ decision discipline

Each row above is something you could be asked about in an interview. The strongest answer is always: "In my PariShiksha system, this concept appeared as a specific decision I made — here is what I chose, here is what it cost, here is the alternative." If you cannot tell that story for a given row, that is a hole worth filling.

4. Environment, data, and tooling

4.1 Hardware (unchanged from Wk9)

- Disk: ~3 GB free (corpus + persisted Chroma collection + cached embeddings)
- RAM: 8 GB minimum, 16 GB comfortable
- Compute: CPU sufficient. No GPU needed for any Wk10 task.
- Python 3.11 in a fresh venv (do not reuse Wk9 venv if it had transient package mess; rebuild clean)

4.2 Pinned versions (cohort standard)

These versions match the trainer demo and the cohort tooling. Pin them in your `requirements.txt`. Version drift between students causes 80% of the Tuesday morning support tickets.

```
langchain==0.3.*
langchain-community==0.3.*
langchain-openai==0.2.*
langchain-anthropic==0.2.*
chromadb==0.5.*
openai>=1.50
anthropic>=0.39
tiktoken
rank_bm25
sentence-transformers
cohere                # optional, for Stretch rerank
ragas                 # optional, for Stretch eval
opendataloader-pdf    # NEW Wk10 (see §4.4)
langchain-opendataloader-pdf # NEW Wk10 (LangChain integration)
python-dotenv
pandas
```

4.3 API access (unchanged shape, new specifics)

- **Generation:** Anthropic `claude-haiku-4-5` by default; `claude-sonnet-4-7` for harder queries (Stretch model-routing demo on Friday). OpenRouter keys distributed via cohort comms — Rushikesh will pin the rotation Monday evening.
- **Embeddings:** OpenAI `text-embedding-3-small` as the cheap default. Free local alternative: `bge-small-en` via `sentence-transformers` (Stretch comparison Wednesday).
- **Reranker:** Cohere `rerank-3` with trial credits — Stretch only. Fall back to `cross-encoder/ms-marco-MiniLM-L-6-v2` from HuggingFace if Cohere is rate-limited.
- **Cost expectation:** Most students stay under \$5 for Wk10. Stretch students running RAGAS may hit \$6–\$8. If you find yourself burning faster than that, your iteration loop is wrong — stop and read §10.

4.4 OpenDataLoader-PDF (new this week)

Wk9 used PyMuPDF or pdfplumber. They worked, but they hit the same wall most beginners hit: tables flattened into walls of text, multi-column reading order broken, equations rendered as mojibake. The teacher's specific complaint about "why does it ignore the answer column" is rooted in this. We need a better loader.

OpenDataLoader-PDF is an open-source PDF parser by Hancom + the PDF Association + Dual Lab (developers of veraPDF). Apache 2.0 licence. Currently #1 in independent benchmarks for AI-ready PDF parsing (DP-Bench, 0.90 overall). Repo: github.com/opendataloader-project/opendataloader-pdf.

Why it fits this week:

- **Outputs structured Markdown** with headings, lists, tables preserved. This is exactly what your structure-aware chunker wants as input.
- **Outputs JSON with bounding boxes** per element (page number + [left, bottom, right, top] in PDF points). The teacher's ask for "specific 200 words" becomes a direct feature.
- **Official LangChain integration** — OpenDataLoaderPDFLoader drops into your existing pipeline.
- **100% local.** No API calls. NCERT PDFs never leave your laptop.
- **Hybrid mode** with a docling-fast backend handles borderless tables and scanned pages.
- **Fast on CPU** — about 60 pages per second.

Quick start:

```
# Core install
pip install opendataloader-pdf langchain-opendataloader-pdf

# Markdown output (best for chunking input)
from langchain_opendataloader_pdf import OpenDataLoaderPDFLoader
loader = OpenDataLoaderPDFLoader(
    file_path="iesc1XX.pdf",          # your chosen NCERT chapter – Motion (iesc107),
                                     # Force and Laws (iesc108), or Gravitation (iesc110)
    format="markdown",
)
docs = loader.load()
# docs[i].metadata = {'source': 'iesc1XX.pdf', 'format': 'markdown', 'page': N}
# docs[i].page_content has Markdown with #, ##, [...] tables, * lists preserved.

# JSON output (when you need bounding boxes for source-highlight UX)
loader_json = OpenDataLoaderPDFLoader(file_path="iesc1XX.pdf", format="json")
elements = loader_json.load()
# Each element has: type, heading_level, page, bbox, content
```

EXPERT HINT · Don't replace your Wk9 pipeline blindly

If your Wk9 PyMuPDF code already handles your chosen chapter cleanly, you don't *have* to switch. The honest test is: does your Wk9 chunker preserve worked-example tables intact? Open your wk9_chunks.json, search for a chapter that has a table, and look at the chunks around it. If the table is broken, switching to OpenDataLoader-PDF on Tuesday is the smallest move that makes the rest of your week saner. If it's intact, keep PyMuPDF and revisit only if Friday evaluation surfaces a structural failure.

4.5 The broader open-loader ecosystem (multiple angles)

OpenDataLoader-PDF is the recommendation for this week. It is not the only option. Knowing the ecosystem matters because in interviews you will be asked "why this loader, not that one?" The honest comparison:

Loader	Strength	Caveat	When to reach for it
OpenDataLoader-PDF	#1 benchmark; bounding boxes; LangChain native; Apache 2.0	Newer (2025); smaller community than alternatives	Default for Wk10. Fits NCERT well.
Docling (IBM)	Strong on scientific PDFs; MIT licence; active community	Heavier dependency footprint	If OpenDataLoader fails on your specific chapter — fall back.
Marker (VikParuchuri)	Excellent on academic papers with equations and references	GPL-3 licence (read this — it affects commercial use of derivative work)	Research/academic corpora. Be aware of the licence if your work goes commercial.

Loader	Strength	Caveat	When to reach for it
Unstructured.io	Very long-standing, broad format support (PDF + DOCX + HTML + email)	Open-core: best features behind their hosted API	When your corpus is mixed-format, not just PDF.
MinerU (OpenDataLab)	Strong on Chinese/Japanese scientific PDFs	Setup heavier; weights are large	Multilingual scientific corpora — not your case this week.
PyMuPDF / pdfplumber (Wk9 default)	Light, fast, predictable	No structure awareness; tables flatten; reading order wrong on multi-column	Already what you have. Keep using if it works on your chapter.

EXPERT HINT · What to actually do

Pick OpenDataLoader-PDF if your Wk9 loader broke tables. Stick with PyMuPDF if it didn't. Try Docling only if OpenDataLoader-PDF gives you trouble on your specific chapter. Do not spend Tuesday afternoon evaluating five loaders — that is a procrastination loop dressed up as engineering.

4.6 Corpus

Same NCERT Class 9 Science chapters as Wk9. Reuse your Wk9 corpus directory. If you only worked on one chapter in Wk9, that's still fine — Wk10's evaluation does not require multi-chapter queries unless you choose them as a Hot Extension.

Source: <https://ncert.nic.in/textbook.php?iesc1=0-11>. Do not commit PDFs to your repo (still). Link from README.

4.7 Repo discipline

- Same repo as Wk9. Tag your Wk9 final commit `v1.0-wk9` Monday before you start Wk10 work. This makes the diff between weeks visible.
- Wk10 work lives on main with feature branches `feat/v2-{day}` for daily work (e.g. `feat/v2-chunking`, `feat/v2-embeddings`).
- Tag `v2.0-wk10` on your final Sunday submission commit.
- `.env` stays out of the repo. `.env.example` with empty placeholders is fine and helpful.
- At least 8 meaningful commits across Tue–Sun (Wk9 hygiene gate was 3; we are raising the bar).

5. The track choice

Tuesday April 28 morning, 09:30, you self-select between Core and Stretch. The selection is a 5-question check, not a test. Honest answers help you. Optimistic answers hurt you.

5.1 The 5-question self-check

#	Question
Q1	Did your Wk9 Study Assistant return non-empty, mostly-relevant answers for at least 7 of 10 test questions you tried?
Q2	Can you explain in 60 seconds out loud what your Wk9 chunking strategy was and why?
Q3	Did you push code with at least 8 meaningful commits across the Wk9 mini-project (not 'wip wip wip')?
Q4	Have you used a vector DB API directly (not only via LangChain abstractions)?
Q5	Are you comfortable reading library source code when documentation is thin?

Eligibility for Stretch: YES to Q1, Q2, and Q3 — plus YES to at least one of Q4 or Q5.

Default to Core if you are uncertain. Mid-week movement is supported in both directions, reviewed Saturday May 2 evening. Anyone who finishes a working v2.0 Core by Thursday and scores above 75% on Saturday's weekly assessment can opt into the Stretch expectations for the Sunday submission. Conversely, Stretch students who miss two consecutive exercise drops shift back to Core. Mon May 4 begins the unified Wk11 production track for everyone who shipped a working v2.0.

5.2 What the two tracks share

- **Same scenario.** PariShiksha, NCERT, the teacher's three asks.
- **Same room, same theory.** Anish runs the morning theory once for both tracks.
- **Same Sunday submission deadline.** May 3, 11:00 PM IST.
- **Same rubric, calibrated to track.** A perfect Core scores the same as a perfect Stretch — the rubric weights are identical; the bar is set per track.

5.3 What the two tracks differ on

Dimension	Core (Foundation Reset)	Stretch (Production Edition)
Vector DBs touched	Chroma (local persistent) only	Chroma + one of Weaviate / Pinecone / Qdrant
Embedding models	OpenAI text-embedding-3-small only	text-embedding-3-small + bge-small-en (local) compared
Retrieval strategy	Top-k=5 dense	Hybrid (BM25 + dense), reranked, MultiQuery enabled
Evaluation	12-Q manual scoring on 3 axes	30-Q golden set + RAGAS report (faithfulness, answer_relevancy, context_precision, context_recall)
Sunday deliverables	Repo + README + 5 sample Q&A screenshots + reflection.md + 3-min Loom	Above + RAGAS report (CSV + memo) + DB-comparison memo + architecture diagram (PNG) + 1-page failure memo + 5-min Loom
Hot Extensions	Not expected (welcome if base ships early)	Optional one of: CRAG / Self-RAG / table extraction / multilingual

Dimension	Core (Foundation Reset)	Stretch (Production Edition)
Faculty time	PM lab: TA + buddy pairing primary; Anish does 2 unblock walkarounds	PM lab: Anish leads; TA handles environment issues

EXPERT HINT · Honesty over optimism on Tuesday morning

Students who pick Stretch when their Wk9 was actually broken end up with two failing weeks instead of one. Students who pick Core when their Wk9 was solid spend the week consolidating and grow more than they expected. Consolidation is undervalued. Pick the track your Wk9 work honestly supports. Saturday's assessment plus the Sat EOD review is when track movement is decided — opt-up is possible there.

6. Core track — five stages

Five stages, executed Tuesday through Friday with polish on Saturday. Each stage produces evidence in your repo (code, data, or written artefact). Submit Sunday May 3 by 11:00 PM IST.

The order matters. You cannot skip stages. Stage 4 (evaluation) is what tells you whether the earlier stages worked. Stage 5 (one targeted fix) only works if Stage 4 was honest.

Stage 1 (Tue) — Repair and structure your chunks

- Tag your Wk9 final commit as `v1.0-wk9` before you start.
- If your Wk9 chunks broke worked-example structure or tables, switch to `OpenDataLoader-PDF` (§4.4) for extraction. Markdown output is the right input to your structure-aware chunker.
- Implement content-type-aware chunking: `prose / worked_example / question_or_exercise`. Use regex on heading and example markers. Keep chunks under ~250 tokens (use tiktoken to size, not character count).
- Add chunk metadata: `{source, section, content_type, page}`. The page field comes free if you used `OpenDataLoader-PDF` JSON output.
- Persist chunks as `wk10_chunks.json`.
- Run BM25 retrieval (your Wk9 retriever) on 5 of your Wk9 eval questions. Log BEFORE (Wk9 chunks) vs AFTER (Wk10 chunks) top-3 results. Note where `content_type` filtering would have helped.

Stage 1 evidence: `wk10_chunks.json` + `chunking_diff.md` (2 paragraphs).

Stage 2 (Wed) — Embed and retrieve

- Embed `wk10_chunks.json` with `OpenAI text-embedding-3-small`.
- Persist to Chroma (`PersistentClient`, path `./chroma_wk10`). Use cosine similarity. Don't re-embed every kernel restart.
- Build `retrieve(query, k=5)` returning chunks with similarity scores.
- Run on 10 of your Wk9 questions. Log top-1 `chunk_id` and a manual YES/NO whether the top-1 actually contains the answer.
- For 3 questions where top-1 was wrong, paste the wrong chunk and write a 1-sentence diagnosis: is this a chunking miss, an embedding limitation, or just bad retrieval ranking?

Stage 2 evidence: `retrieval_log.json` + `retrieval_misses.md`.

Stage 3 (Thu) — Wire generation with strict grounding

- Wire your retriever to Anthropic `claude-haiku-4-5` at `temperature=0`.
- Start with a *permissive* prompt ("Answer the question using the context"). Run on 3 queries including one out-of-scope. Log the result. Expect at least one hallucination — that's the lesson.
- Rewrite the prompt as a *strict* prompt: "If the answer is not present, reply exactly: 'I don't have that in my study materials.' After every factual claim, cite the chunk it came from in square brackets, e.g. [Source: `chunk_id`]."
- Re-run the same 3 queries. Hallucination should resolve to clean refusal. Save both responses verbatim in `prompt_diff.md`.
- Update `ask(question)` to return `{answer, sources, chunk_ids}`.

Stage 3 evidence: `ask()` in your repo + `prompt_diff.md`.

Stage 4 (Thu evening through Fri morning) — Evaluate honestly

- Build a 12-question evaluation set: 6 direct + 3 paraphrased + 3 out-of-scope. Include at least one "plausibly answerable" out-of-scope question (e.g., "Calculate g on the surface of the Moon" — formula is in your corpus, Moon-specific values aren't).
- Run all 12 questions through ask(). Save raw output as `eval_raw.csv`.
- Score each question by hand on three axes: (a) *correct* Y/N/partial, (b) *grounded* Y/N — citation present and the cited chunk actually contains the claim, (c) *refused_when_oos* Y/N/NA. Save as `eval_scored.csv`.
- Hand-scoring 12 takes about 12 minutes. Do not skip this for an LLM-judged version. The judgement calls you make are part of your engineering practice.

Stage 4 evidence: `eval_scored.csv` + a 1-paragraph diagnosis on the worst-performing question.

Stage 5 (Fri) — One targeted fix, with honest delta

- Pick the **single worst** failure in your Stage 4 eval (wrong answer, ungrounded, ideally out-of-scope-but-not-refused).
- Diagnose its category from the failure-mode catalog (§3): synonym/acronym mismatch, lost cross-reference, multi-hop, mixed structure, ambiguous query.
- Implement *one* targeted fix matching the diagnosis: a stricter prompt phrasing, or a structure-aware chunk for that section, or a metadata filter at retrieval. Do not try three fixes at once.
- Re-run the full 12-Q eval with the fix in place. Save as `eval_v2_scored.csv`.
- Write `fix_memo.md`: which fix, why this fix targeted this failure, what the score delta was. **Honest delta.** If the fix didn't help or hurt other questions, say so. Reflection counts.

Stage 5 evidence: `eval_v2_scored.csv` + `fix_memo.md`.

EXPERT HINT · Why one fix, not three

When you change three things at once and your eval moves up by 2 points, you do not know which change helped and which hurt. You also do not know whether you'd have gained 4 points with only one of them. Single-variable iteration is slower this week and faster every week after. Anish will check that your git log shows this discipline. So will your future tech lead.

7. Stretch track — five stages

Same five-stage shape as Core, but each stage adds production-shaped complexity. You produce more artefacts. Your evaluation is broader and partly automated. You commit to one specific production pattern beyond the basic pipeline.

Stage 1 (Tue) — Two chunking variants, with measurement

- Same Stage 1 as Core, plus implement a *second* chunking variant (semantic chunking via `langchain_experimental.text_splitter.SemanticChunker`, or a custom heading-anchored splitter using `OpenDataLoader-PDF`'s heading metadata).
- Hand-build a 10-question micro-eval set. Run BM25 retrieval with both chunking variants. Score top-1 hit rate per question.
- Write `chunking_compare.md` (1 page): which variant won, where each fails, what you'd carry into Stage 2.

Stage 2 (Wed) — Two vector DBs, two embedding models, real benchmark

- Index your chunks in Chroma *and* one of Weaviate Cloud / Pinecone / Qdrant.
- Embed with two models: `text-embedding-3-small` + `bge-small-en` (local, via `sentence-transformers`).
- Run 20 queries on each (DB × model = 4 combinations). Capture latency p50 and p95 per combination + `recall@5` (manual judgement).
- Save as `db_benchmark.csv`.
- Write `db_comparison.md` (1 page): which combination won, why, what would change at 10× scale (consider write throughput, query latency under concurrent load, cost per million tokens, cold-start vs warm).

Stage 3 (Thu) — Hybrid retrieval + strict grounding + 20-Q eval

- Implement hybrid retrieval: BM25 + dense, fused by reciprocal rank fusion (RRF) or weighted scores. Use the `LangChain EnsembleRetriever` as your starting point.
- Wire to `claude-haiku-4-5` with the strict prompt (refusal + citation format).
- Expand eval set to 20 questions (10 direct + 5 paraphrased + 5 out-of-scope, with at least 2 "plausibly answerable" out-of-scope queries).
- Score on the 3 axes. Save as `eval_v3_scored.csv`.
- Identify 2 distinct failure patterns. Write 1 sentence each on the most likely cause.

Stage 4 (Fri) — Add rerank + MultiQuery + RAGAS report

- Add Cohere rerank-3 on the top-20 → top-5 path. Wire a fallback to `cross-encoder/ms-marco-MiniLM-L-6-v2` in the same code path so a Cohere rate-limit doesn't kill your Sunday submission.
- Add `MultiQueryRetriever` — Anthropic Haiku rewrites each query into 3 variants. Pool results, dedupe, rerank against the original query.
- Build a 30-question golden set (10 easy + 15 medium + 5 hard, mixed direct/paraphrased/OOS).
- Run RAGAS with Haiku-as-judge. Capture *faithfulness*, *answer_relevancy*, *context_precision*, *context_recall*. Save as `ragas_report.csv`.

- **Cost discipline:** RAGAS judge tokens $\approx 4 \text{ metrics} \times 30 \text{ Q} \times \sim 2\text{k tokens} \approx \1.50 on Haiku. Stop after the first run. Don't re-run RAGAS every time you tweak — wasteful and the signal-to-noise drops fast.
- Target: *faithfulness* ≥ 0.7 . If below, the diagnosis is almost certainly retrieval, not generation.

Stage 5 (Fri evening / Sat) — Failure memo + architecture diagram

- Write `failure_memo.md` (1 page): the top three failure modes your v2.0 still has, grounded in your actual eval data. For each: where it fails, why, and the smallest fix you'd make in Wk11. Speculation flagged as speculation.
- Produce `architecture.png`: a clear architecture diagram of your v2.0. Boxes for ingestion $\rightarrow$ chunking $\rightarrow$ embedding $\rightarrow$ vector store $\rightarrow$ BM25 $\rightarrow$ fusion $\rightarrow$ rerank $\rightarrow$ MultiQuery $\rightarrow$ ask() $\rightarrow$ eval. Arrows show data flow. Annotations show key choices (model names, k values, fallback paths). Use any tool — draw.io, Excalidraw, Figma, even hand-drawn-and-photographed is fine if legible.
- **Stage 5 evidence:** `failure_memo.md` + `architecture.png`.

EXPERT HINT · Stretch is not 'more features'. Stretch is 'more honest measurement.'

The students who do badly on Stretch are the ones who add features without measuring them. Adding rerank + MultiQuery and not running the eval after is wasted work and a missed interview talking point. The whole reason to add these features is to *show* the lift (or absence of lift) on real questions. Stretch grading weights eval quality more than feature count.

8. Hot Extensions (optional, after your base ships)

Hot Extensions exist for students who finish their base track Friday afternoon and have bandwidth Saturday and Sunday. They are **optional**. Anish does not review failed Hot Extension attempts in Wk10. Ungated. Push failures honestly — failures count for Wk11 discussion and capstone scope decisions, but they do not affect your Wk10 score.

Pick at most one. Time-box it. If it isn't working by Saturday evening, document what you tried and stop. Polishing your base is more valuable than half-shipping an extension.

Extension	What you build	Effort	Why it's worth it
CRAG (Corrective RAG)	Retrieve → grade chunk relevance with a small LLM call → if low confidence, rewrite query and retry once. Implement the grader as a separate function so you can swap it out.	4–6 hrs	Demonstrates conditional retrieval logic. Bridges to LangGraph (Sun teaser). Real interview talking point on "how do you handle bad retrievals?"
Self-RAG	Add a 'do I need to retrieve at all?' reflection step before retrieval. The model decides retrieval-needed vs retrieval-skip based on the query.	5–8 hrs	Demonstrates retrieval gating. Useful when 30% of real-world queries don't actually need retrieval (chitchat, definitional, formulaic).
Table extraction	Use OpenDataLoader-PDF JSON mode (or pdfplumber as fallback) to extract NCERT tables as separate chunks with structured metadata. Show retrieval works on a table-specific question.	3–5 hrs	Directly addresses the teacher's complaint about ignored answer columns. Most production-relevant of the four.
Multilingual / code-switched query handling	Test 5 queries in Hindi (Devanagari), 5 in code-switched English-Hindi ("newton ka second law kya hai?"). Report retrieval and grounding behaviour.	2–4 hrs	PariShiksha's actual user base does this. Shows you thought about the deployment context, not just the textbook.
Cite-and-quote mode	Modify ask() to return exact source quotes (≤15 words each) with chunk_ids and bounding boxes (using OpenDataLoader-PDF JSON), not just summaries.	3 hrs	Directly delivers the teacher's third ask. The 'click to source' UX foundation.

EXPERT HINT · Push to feat/hot-ext branch — never main

Hot Extensions stay on a feature branch. Do not merge them into main until you can demonstrate they work end-to-end. Half-shipped CRAG on main is worse than no CRAG — it confuses the rubric grader and obscures your honest base submission.

9. Industry Pointers (awareness, not deliverables)

These are the patterns real production teams use beyond what we build this week. They are **not** for student build this week. They are here so you can speak about them in interviews and recognise them in capstone scoping (Wk11 onward). Pick one to mention in your `reflection.md` as something you'd explore in a 6-month timeline — that earns the +3 awareness bonus.

Pattern	What real teams do	Why it matters at scale
Domain-fine-tuned embeddings	Fine-tune a sentence-transformer on (query, positive_passage, negative_passage) triplets from your domain logs. Use multi-task negatives or hard negatives for harder gains.	Quality lift can exceed swapping to a 5× larger generic model. Standard in legal / medical / scientific / finance retrieval.
Layout-aware chunking at industrial scale	Use Unstructured.io / Docling / OpenDataLoader-PDF with layout-aware mode on PDF pipelines. Enforce structure-tag preservation in regulated domains.	PDF-heavy domains where reading order, table relationships, and figure-caption pairing matter. Compliance often demands this.
Multi-vector retrieval (ColBERT)	Each token gets its own embedding. Late interaction matches query tokens to document tokens. Tools: ColBERTv2, PLAID, RAGatouille.	Stronger on long passages and exact-term recall. Higher infra cost and storage. Used by Vespa, Elastic, and serious search infra teams.
Fine-tuned cross-encoder reranker	Train a cross-encoder on (query, passage, relevance_label) from your own labelled data. SBERT trainer or HuggingFace's CrossEncoder API.	Cohere rerank-3 is generic. Domain rerankers can lift recall@5 by another 5–10 points on domain queries.
Prompt versioning and A/B testing	Store prompts in PromptLayer / LangSmith hub. A/B test prompt versions on production traffic. Roll back the way you'd roll back code.	Prompt drift is a real bug class. A 0.3 quality drop because someone tweaked a prompt at midnight is a real outage you want to catch and revert.
Grounding-aware decoding	Constrained decoding (Outlines, Guidance, JSON mode with citation schema) forces the model to cite retrieved chunks structurally, not just by instruction.	Hard guarantees, not soft. Used where hallucination is a regulatory issue (medical, legal, finance, education at scale).
Hybrid retrieval at production scale	BM25 (Elasticsearch / OpenSearch) + dense (Vespa / Qdrant / pgvector) with a learned fusion layer. Often a small neural fusion classifier.	Production RAGs at >10M docs almost always run hybrid. Pure-dense at that scale is too expensive. Pure-BM25 misses paraphrase queries.
Eval-driven development	Treat eval set as code. Version it. Add a question every time you find a real failure. Run eval in CI on every retrieval-touching PR.	The eval set is the moat. It separates a system that ships from a system that hallucinates in production. Most teams underinvest here.
Tenant isolation at the index level	tenant_id as enforced metadata. Multi-tenant systems use namespace-per-tenant or filter-at-index, not filter-at-query.	B2B SaaS RAG. One customer's chunks must never leak into another customer's retrieval. Filter-at-query is a CVE waiting to happen.
Multi-modal retrieval	When documents have images, charts, diagrams: CLIP embeddings, multimodal models (Claude vision, GPT-4o vision), image-text hybrid retrieval.	Pure text RAG fails silently on visual content. Capstone-relevant if your domain has diagrams (medical imaging, blueprints, scientific figures).

EXPERT HINT · How to use this section in interviews

When an interviewer asks "how would you take this to production?" the answer that lands is rarely "I'd add CRAG." The answer that lands is "my evaluation showed grounding failures concentrated in paraphrased queries — at production scale I'd invest in domain-fine-tuned embeddings or a fine-tuned cross-encoder reranker first, because that's where the lift is for *my* failure mode." Specific. Grounded in measurement. Aware of cost. That's what a senior engineer sounds like.

10. Expert hints

These are the things a senior RAG engineer would tell you across a table over coffee. They are not in the standard tutorials. Each one matches a mistake most people make on their first or second build. Read them once now. Read them again Friday evening when something is broken.

EXPERT HINT · Print retrieved chunks before blaming the model — say it again on Wednesday

Anish will repeat this principle daily. Believe it. When v3 hallucinates Thursday, your reflex will be "swap to Sonnet, the prompt isn't strong enough, change the model." Wrong nine times in ten. Add one print statement to your `ask()` that dumps the top-k retrieved chunks before the model is called. Read what came back. Most Thursday hallucinations resolve with that print in under five minutes.

EXPERT HINT · Index-time and query-time embeddings must come from the same model

If you embed your chunks with `text-embedding-3-small` but accidentally call openai's `text-embedding-ada-002` for queries (because some old code path leaked in), every cosine similarity is meaningless and you won't notice — you'll just get bad retrieval. Lock the embedding model in one place, import it everywhere, never re-instantiate.

EXPERT HINT · Pin `chromadb==0.5.x` — explicitly

Chroma's persistence format changed at 0.4 and again subtly at 0.5. If a teammate has 0.4.x and you have 0.5.x, your collection won't load on their machine. The fix: `chromadb==0.5.*` in `requirements.txt`. Tuesday-morning support tickets are 80% version drift, 15% `.env`, 5% real bugs.

EXPERT HINT · Don't re-embed on every kernel restart

OpenAI `text-embedding-3-small` on a 30-page chapter takes about 90 seconds to embed and costs ~\$0.10. If your notebook re-embeds on every cell run because you didn't gate it on `collection.count() == 0`, you'll burn \$5 by Friday and lose 30 minutes per restart. Embed once. Persist. Check existence before re-embedding.

EXPERT HINT · Top-k=10 is not automatically better than top-k=5

More chunks in context means more tokens, more latency, more cost — and crucially, the model spends attention on lower-relevance chunks at the bottom of the list. A k=10 retrieval where ranks 6–10 are noise produces worse answers than a k=3 retrieval. Read the similarity scores. If rank 5's score is 0.40 and ranks 6–10 are below 0.30, drop them. The bar is "does this chunk help", not "does this chunk fit in context".

EXPERT HINT · Where dense retrieval loses to BM25 — and why hybrid is the answer

Dense embeddings shine on paraphrased meaning. They lose on exact identifiers, acronyms, numeric tokens, code symbols, scientific notation. "`g = 9.8 m/s2`" is a string BM25 finds trivially and dense often misses. The production move is hybrid — both retrievers, fused. Stretch students do this Thursday. Core students should at least understand why the trainer demo Wednesday afternoon shows BM25 winning two of five queries.

EXPERT HINT · "Refuse if not in context" is a constraint, not a hint

The Wk9 hint was: "Only answer from context" is permissive; "Refuse if not in context" is a constraint. Wk10's specific test of this: build a prompt that says *both*. ("Answer only from the context. If the answer is not in the context, reply exactly: 'I don't have that in my study materials.'") The first sentence alone fails 30% of the time on plausibly-answerable out-of-scope queries. Two sentences together fail closer to 5%. Test it.

EXPERT HINT · Cite chunk_ids in the answer — even if the user never reads them

{answer, sources, chunk_ids} is not just for the teacher's highlighting demand. It is the only way you, the engineer, debug a wrong answer. Without the chunk_ids in your response object, you cannot tell whether the model invented or whether retrieval handed it the wrong source. With them, every wrong answer becomes a 30-second diagnosis.

EXPERT HINT · RAGAS faithfulness ≠ correctness

Faithfulness asks: "is the answer supported by the retrieved context?" An answer that says "the moon is made of cheese" can be 1.0 faithful if your retrieved chunk says it, and 0.0 correct because the chunk was wrong. RAGAS faithfulness measures grounding, not truth. If your Stretch Friday RAGAS report shows faithfulness 0.85 and you celebrate, look at *context_precision* next — that's whether your retriever returned relevant chunks at all.

EXPERT HINT · Reranking has a latency cost — show it, don't hide it

Cross-encoder rerank doubles your retrieval latency (bi-encoder pass + cross-encoder pass). On 1M+ daily queries that matters. Stretch students will not feel the cost at student scale, but interviewers ask about it. Measure it: time the retrieval step before and after rerank, report both numbers in your *db_comparison.md* or *fix_memo.md*. Talking about a rerank without knowing the latency it cost you is the kind of answer that signals "hasn't shipped".

EXPERT HINT · Eval set quality is the moat

A bad eval set hides a bad system. The temptation Thursday will be to write 12 questions you *know* your system can answer. Don't. Include the questions you suspect break things. Include the "plausibly answerable out-of-scope" question (the Mars-gravity case). Real production teams add a question to the eval every time they find a failure in production. The eval set grows; the system stays honest. Your Wk10 eval is small but it sets the habit.

EXPERT HINT · Single-variable iteration — your git log is your experiment log

When eval results disappoint Friday afternoon, you'll be tempted to swap the model, rewrite the prompt, and add MultiQuery in the same hour. Don't. Each change gets its own commit with a message that names the change. "Swap reranker to cross-encoder fallback (Cohere rate-limited)." "Add MultiQuery — 3 variants per query." Your git log becomes the experiment log. When viva asks "why did your faithfulness drop between Friday morning and Friday evening?", git answers in 30 seconds.

EXPERT HINT · Pre-build the rerank fallback — Cohere will rate-limit you when you need it most

Free-tier Cohere credits are generous but not unlimited. If you wire only Cohere rerank with no fallback, your code starts failing at the worst possible moment — usually peak cohort traffic Friday afternoon during lab or Sunday evening just before submission. The trainer demo handles this with a try/except that drops to cross-encoder/ms-marco-MiniLM-L-6-v2 from HuggingFace (free, runs locally on CPU, good enough for student-scale work). Wire the fallback first. Test it before you need it.

EXPERT HINT · Sunday submission rate-limit reality

Anthropic and OpenAI APIs spike with cohort traffic on Sunday evenings 9–11 PM IST. If you are running RAGAS at 10:30 PM Sunday for the first time, expect 429s. The fix: run RAGAS Saturday afternoon. Save the report. Sunday is for polishing your README and Loom, not for first-time API calls.

EXPERT HINT · Your Loom is part of your submission, not an afterthought

Three minutes (Core) or five minutes (Stretch). Show your repo, show ask() running on three queries (one in-scope, one paraphrased, one out-of-scope-and-refused), show your eval table. No screen-reading. Speak to camera. Practice once before recording. The Loom is what the grader sees first; if it's confused or quiet, your repo gets read with that prior. Strong Looms get strong reads.

11. What "done" looks like + rubric + submission

11.1 Core complete

A runnable repo with: (a) Tue chunks with metadata persisted as `wk10_chunks.json`, (b) Wed Chroma collection populated with `text-embedding-3-small` embeddings, (c) Thu `strict-grounding ask()` returning `{answer, sources, chunk_ids}`, (d) 12-Q eval table scored on three axes showing at least 8 correct + 9 grounded + 3 of 3 OOS refused, (e) Fri one targeted fix with honest before/after eval delta, (f) all six evidence files (`chunking_diff`, `retrieval_misses`, `prompt_diff`, `eval_scored`, `eval_v2_scored`, `fix_memo`) committed.

11.2 Stretch complete

Core complete plus: (a) two chunking variants compared, (b) Chroma + one cloud DB benchmarked, (c) hybrid retrieval (BM25 + dense fusion), (d) rerank with cross-encoder fallback, (e) MultiQueryRetriever wired, (f) 30-Q golden set with RAGAS report (faithfulness ≥ 0.7), (g) `failure_memo.md` + `architecture.png`.

11.3 Rubric (100 points + bonus)

#	Criterion	Weight	What's assessed
1	Chunking and corpus structuring	15	Content-type metadata applied. Token-aware sizing. Worked-example tables preserved. Stretch: chunking variants compared with measurement.
2	Embedding and retrieval	15	Chroma persistence works. Top-k retrieval honest. Stretch: hybrid retrieval, DB benchmark, embedding model comparison.
3	Grounded generation	15	<code>ask()</code> refuses cleanly on OOS. Citation format consistent. Strict prompt phrasing. Stretch: model routing.
4	Evaluation quality	20	Eval set is honest (includes plausibly-answerable OOS). Three axes scored. Stretch: 30-Q + RAGAS with cost discipline.
5	One targeted fix with honest delta (Core) / production patterns layered (Stretch)	15	Single-variable iteration visible in git log. Honest before/after numbers. Stretch: rerank + MultiQuery + structure-aware combined.
6	Reflection questionnaire	10	Specific to your artefacts. References real <code>chunk_ids</code> , real eval rows, real git commits. LLM-fluent generic answers score lower than honest specific ones.
7	Repo discipline + Loom	10	8+ meaningful commits. README runs on fresh clone. Loom shows the system working live.
	Total	100	
+	Industry Pointer awareness (mention one in <code>reflection.md</code> as a 6-month exploration target)	+3	
+	Hot Extension attempted with honest results (Stretch only)	+5	

11.4 Submission (Sun May 3, 11:00 PM IST)

Submit a public GitHub repository link via your assigned GitHub Discussions thread. Repository must contain:

- `README.md` — setup, run instructions, link to NCERT source (not committed)
- `requirements.txt` — pinned versions
- `notebook.ipynb` or `src/` with end-to-end runnable code
- `wk10_chunks.json` + `chunking_diff.md`
- `retrieval_log.json` + `retrieval_misses.md`
- `prompt_diff.md`
- `eval_scored.csv` + `eval_v2_scored.csv`
- `fix_memo.md`
- `reflection.md` (Section 12 of this document)
- Loom link in README (3 min Core / 5 min Stretch)
- **Stretch only:** `db_comparison.md`, `chunking_compare.md`, `ragas_report.csv`, `failure_memo.md`, `architecture.png`

WARNING · Submission hygiene gate (PASS/FAIL — same as Wk9)

Required files present. Notebook (or `src`) runs on a fresh clone with `.env` set. At least 8 intermediate commits in git history. Loom link present and unlisted-but-accessible. Missing any of these zeroes the project regardless of experiment quality. Reproducibility is a real skill we are testing.

Late submission: follow the standard cohort late-submission policy. If you have a genuine reason you cannot meet Sunday 11:00 PM IST, message Akash directly *before* the deadline. After-the-fact requests rarely get accommodation.

12. Reflection questionnaire

Submit this as `reflection.md` in your repo. Target length: 900–1200 words across all answers. Honesty over polish.

EXPERT HINT · About this questionnaire

These questions reference your specific artefacts: your chunk IDs, your eval rows, your git commits, your specific failure cases. An LLM cannot tell the grader which `chunk_id` had the highest similarity score and was wrong, because that information lives only in your repo. Don't fight the questions — answer them. Short, specific, honest beats long, vague, polished.

Part A — Your implementation specifics

These ask about lines of code and files in your repo. Cannot be guessed.

A1. Chunking decisions, with evidence

Paste your final chunking parameters (`chunk_size` in tokens, overlap, content-type splitter rules). Then: list *three* specific chunks from your `wk10_chunks.json` by `chunk_id` and explain what `content_type` each one got and why. At least one must be a `worked_example` or `table` chunk.

A2. The chunk that surprised you

Paste one chunk from your collection (`chunk_id` + first 200 chars) where you expected one `content_type` assignment and got another. Explain what the heuristic missed. If your heuristic made no mistakes, paste a chunk where it got it right by accident — there's almost always one.

A3. Loader choice

Did you stay on PyMuPDF/pdfplumber from Wk9 or switch to OpenDataLoader-PDF? Name *one* specific behaviour difference you observed on your chapter — better, worse, or just different. If you didn't observe a difference, that's a useful answer too: explain how you tested.

Part B — Numbers from your evaluation

B1. Eval scores, raw

Out of N questions in your `eval_scored.csv`: (a) how many correct, (b) how many grounded, (c) how many appropriate refusals (for OOS only)? Then: which of these three numbers bothered you most, and why?

B2. The single worst question

Paste the question text. Paste the answer your system gave. Paste the top-3 retrieved `chunk_ids`. In two sentences: what was the actual failure mode (use the §3 catalog: synonym mismatch / lost cross-ref / multi-hop / mixed structure / ambiguous)?

B3. Before-and-after on your one fix (Core) or RAGAS (Stretch)

Core: What was your single targeted fix? What was the eval delta — both on the specific question you targeted, and across the full 12-Q set? If the fix didn't help, what does that tell you?

Stretch: Paste your RAGAS scores (faithfulness, answer_relevancy, context_precision, context_recall). Which metric is highest and which is lowest? What does that tell you about where your system fails — retrieval or generation?

Part C — The 30-second debugging story

C1. The retrieved chunk that fooled you

Find one query in your eval where the top-1 retrieved chunk *looked* right (high similarity score, plausible content) but actually did not contain the answer. Paste the query, the chunk text (first 250 chars), and the score. Then: why do you think it ranked top-1? Be specific — "the query and chunk share keyword X but X means something different in each".

C2. The bug that took you longest

Describe the single bug that ate the most of your week. How long did it take to fix? What did you try first that didn't work? What was the actual fix? If a teammate hits the same bug next week, what's the fastest path to the fix?

C3. The thing that still bothers you

Even if your v2.0 is mostly working, name one specific behaviour that still bothers you. Why? What would you try in Wk11 to address it?

Part D — Architecture and tradeoffs

D1. Why hybrid retrieval (or why not)?

Your hiring manager asks: "Why do we need both BM25 and dense retrieval? Pick one and ship." Respond using *one specific query* from your eval where one retriever clearly beat the other. Include both retrieval results.

D2. The CRAG / Self-RAG question

You heard about CRAG and Self-RAG awareness-only on Friday. In your own words, when would you build CRAG in production, and when is it overkill? Reference your *own* failure modes — would CRAG have helped your worst eval failure? Why or why not?

D3. Honest pilot readiness

PariShiksha's CTO asks: "Can we launch this v2.0 to 100 students next Monday?" Based on what your system actually did in evaluation, what's your honest answer? Name three specific things you'd want to verify or fix first. Reference real eval rows, not hypotheticals.

Part E — Effort and self-assessment

E1. Effort rating

On a scale of 1–10, how would you rate your effort on this project? If below 7, what got in the way (be specific — ME1 fatigue, environment issues, unclear goals)? If 7+, name one specific thing you did this week that you are genuinely proud of.

E2. The gap between you and a stronger student

Imagine a stronger student in the cohort working on the same project. Name one specific thing they probably did that you didn't. Is there a reason you didn't (time, confidence, didn't understand the value)?

E3. The Industry Pointer you'd explore in 6 months

Pick one entry from §9 (Industry Pointers). Why this one for your trajectory? What would the first concrete step be? (This earns the +3 awareness bonus.)

E4. Two more days

If you had exactly two more days on this project, what's the *first* thing you'd do, and what's the *last* thing you'd do? Order matters — it tells the grader what you actually think is most important.

13. Interview-flavor scenarios

Practise these out loud with a teammate, in front of a mirror, on a walk. They are case-study questions — there is no single correct answer. The interviewer is evaluating *how you think*, not what you memorised. The strongest answers reference your specific PariShiksha system and your actual eval results.

EXPERT HINT · How to approach case-study questions without anxiety

Three things land well: (1) name the thing you'd need to know before deciding, (2) walk through your reasoning aloud even when uncertain, (3) acknowledge what you don't know rather than bluff. Interviewers reward honest thinking over confident guessing. If you have actually built v2.0 this week, you have real experiences to draw from. Use them. Specific is stronger than impressive.

Scenario 1 — The 'almost right' diagnosis

"Your PariShiksha assistant gave the teacher 8 'almost right' answers out of 30. As the engineer, where do you start your investigation? Walk me through your debugging in order."

What the interviewer wants: ordered diagnosis, not panic. Strong answers say: "First I print the retrieved chunks for the 8 failures and check whether the right chunk was even in top-10 — that splits the bug into 'retrieval missed the right chunk' vs 'retrieval found it but generation didn't use it'. Then I look at the prompt and citation format. Last I look at the model." Reaching for "swap to a smarter LLM" first is the wrong-instinct answer.

Scenario 2 — The 'why hybrid' interview

"Your CTO says: 'BM25 is from 1994. Why are we still using it? Just go pure-dense and save the engineering complexity.' Defend the call you'd make."

What the interviewer wants: evidence-backed defence with named failure modes. Reference a specific query where dense lost to BM25 in your eval (acronym, exact identifier, formula, numeric token). Note that hybrid handles both classes. Acknowledge the operational complexity is real and worth it. Strong candidates also note that pure-dense at 10M+ docs is expensive — fusion gets you most of the recall at a fraction of the cost.

Scenario 3 — The chunk-size tradeoff

"You have 250-token chunks. Someone proposes 1000-token chunks because 'more context, better answers'. Walk me through the tradeoffs."

What the interviewer wants: three vectors of awareness. **Retrieval:** bigger chunks dilute embedding signal — the chunk's vector represents an averaged "meaning" across more content, fewer chunks rank well. **Generation:** bigger chunks eat context window and the model's attention spreads thinner. **Citation:** bigger chunks make citation less useful — "answer is in chunk_42" is less actionable when chunk_42 is 1000 tokens. Strong answers also note structure-aware chunking (semantic boundaries) often beats both extremes.

Scenario 4 — The domain-shift question

"A pharmaceutical company wants a similar RAG over FDA regulatory filings. What three changes from your PariShiksha build matter most? Rank them."

What the interviewer wants: ability to reason from context to architecture. Strong answers identify: (1) *citation precision is now liability-grade* — bounding boxes on every claim, OpenDataLoader-PDF style; (2) *cross-document reasoning* — FDA queries span filings, this needs multi-hop or query decomposition; (3) *domain-fine-tuned embeddings* because regulatory language is specialised; (4) *much stricter OOS handling*

— the cost of a wrong answer is a lawsuit, not a refund. Great answers rank these and defend the ranking with a specific failure mode.

Scenario 5 — The scale question

"Your v2.0 works for one school with one chapter. Walk me through what breaks first as we go to 500 schools, 50 chapters each, 50,000 queries per day."

What the interviewer wants: production thinking. Touch on: *concurrent latency* — your local Chroma cannot serve 50k qps; you move to Pinecone or Vespa or Qdrant cluster. *Cost per query at scale* — embedding API and LLM bills become real; cache aggressively, consider local embeddings for hot paths. *Eval drift* — real student queries diverge from your hand-picked 12; you need ongoing eval-set growth from production logs. *Monitoring* — how would you know if grounding rate dropped 5% next month? *Multi-tenancy* — student data isolation, school_id as enforced metadata.

Scenario 6 — The 10-engineering-hours tradeoff

"You have 10 additional engineering hours. You must pick exactly one: (a) better chunking, (b) tighter grounding prompt, (c) add reranking, (d) add explicit OOS guardrails. Which, and why?"

What the interviewer wants: decision-making under constraint, anchored to *your* system's failure modes. The right answer depends on what your eval actually shows: "my eval showed 7 of 12 failures were retrieval misses (right chunk not in top-5), so I'd pick (c) reranking — the right chunk is being found but ranked low." Weak answers pick a tier without justification. The strongest answers also mention a measurement plan: "and I'd re-run my 30-Q golden set after to confirm."

Scenario 7 — The whiteboard architecture extension

"Draw your PariShiksha architecture on the whiteboard. Now: students often code-switch between English and Hindi (devanagari). Draw what changes."

What the interviewer wants: ability to extend an existing design to a new requirement. Strong answers identify: (1) tokeniser behaviour on Devanagari differs from English; (2) embedding models trained primarily on English may degrade — multilingual models like multilingual-e5-base or BAAI/bge-m3 exist; (3) eval set must include code-switched queries; (4) chunking heuristics may need language detection. Great answers also note multilingual reranker availability.

Scenario 8 — The production failure call

"Your assistant has been live for two weeks. A parent calls, furious — the assistant told her daughter that Newton's second law is $F = mv$ instead of $F = ma$. How do you investigate? What do you tell the parent in the meantime?"

What the interviewer wants: ops thinking and incident discipline. Sketch a debugging playbook: *reproduce* the specific query (you should be logging queries + responses + chunk_ids), *inspect* what chunks were retrieved (was the wrong formula in the corpus? was the right one ranked below it?), *determine* retrieval miss vs generation error, *mitigate* immediately (add to eval, manual override on this query, block deployment until fix passes), *root-cause* fix. To the parent: acknowledge, do not over-promise, commit to a follow-up time.

Scenario 9 — The eval-set integrity question

"Your 30-Q golden set shows 95% faithfulness and 90% answer-relevancy. The CTO is delighted. Should they be?"

What the interviewer wants: healthy scepticism and eval-set awareness. Strong answers: (1) ask who built the golden set — if you wrote it yourself reading the textbook, your queries skew towards what your retriever can find; (2) note that *context_precision* matters more than faithfulness here — high faithfulness on irrelevant chunks is still wrong; (3) ask about real-user query distribution — are golden-set queries representative of

what students actually type? Production traffic always reveals failure modes the golden set hides. Eval-driven development is a habit, not a one-shot.

14. One closing thought

PariShiksha is not decoration. The teacher is not a fictional persona. She represents the specific kind of user whose trust your system has to earn — someone with deep domain knowledge, high standards, and zero tolerance for confident-sounding wrongness. Every choice in this document — the strict prompt phrasing, the citation requirement, the structure-aware chunking for tables, the OOS-and-refused test cases — exists because users like her exist.

If you build your v2.0 by Sunday and it can answer the teacher's 30 questions with measurable grounding, you have done something most engineers shipping LLM products in 2026 cannot. The eval discipline you practise this week is what separates an engineer who ships from an engineer who demos. The reflection questionnaire is what tells future-you the difference.

Take the evaluation seriously. Write the strict prompt carefully. Let your system fail on something specific and trace the cause to one of the five catalog categories. Print the retrieved chunks before blaming the model.

That is the work.

— End of project document —